NAME : G. AKSHAYA **COURSE CODE** : CSA0619

REG NO: 192521156 **COURSE NAME** : DESIGN AND ANALYSIS OF ALGORITHMS

CO2 - ASSESSMENT TOOL – 03

Q2. Debugging Bubble Sort Code – Swap Flag Not Reinitialized

Problem Analysis

The given Bubble Sort implementation attempts to use a swapped flag to improve efficiency by stopping the algorithm when the array is already sorted.

However, there are several problems in the given code:

Errors identified

1. Swapped is initialized only once

   swapped = False

It should be reset to False at the beginning of **every outer-loop pass**.

2. Break is incorrectly placed inside the swapping condition

   if arr[j] > arr[j+1]:

   arr[j], arr[j+1] = arr[j+1], arr[j]

   swapped = True

   break

This causes the inner loop to stop immediately after the first swap. Therefore, the remaining adjacent elements are not compared during that pass.

3. Incorrect return variable

   return array

The parameter is named array, not array. Therefore, this causes a Name Error.

4. No proper return at the end

   The sorted array should be returned after all passes are completed.

Effect of the errors:

Because the flag is not reinitialized correctly and the break terminates the inner loop after the first swap, the algorithm may:

- Stop before completing a complete pass.
- Fail to sort the array correctly.
- Continue unnecessarily in some situations.

- Produce an error because array is undefined.

2. Step-by-Step Debugging

Consider the input:

[5, 4, 3, 2, 1]

Incorrect logic

Initially:

swapped = False

During the first pass:

5 > 4 → Swap

[4, 5, 3, 2, 1]

Then:

break

immediately stops the inner loop.

So the algorithm does not continue comparing:

5 and 3

5 and 2

5 and 1

This means one complete Bubble Sort pass has not actually been performed.

Correct logic:

At the beginning of every pass:

swapped = False

During the pass, whenever a swap occurs:

swapped = True

The inner loop must continue until all adjacent elements have been checked.

After the complete pass:

if not swapped:

break

This means:

If no swaps occurred during the entire pass, the array is already sorted, so terminate early.

3.Corrected and Optimized Source Code

main.py

```python
def bubble_sort(arr):
    n = len(arr)
    # Perform multiple passes
    for i in range(n - 1):
        # Reset swap flag for every pass
        swapped = False
        # Compare adjacent elements
        for j in range(0, n - i - 1):
            # Swap if elements are in the wrong order
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        # If no swapping occurred, array is already sorted
        if not swapped:
            break
    return arr
# Input
arr = [64, 25, 12, 22, 11]
# Sorting
sorted_array = bubble_sort(arr)
# Output
print("Original/Sampled Array: [64, 25, 12, 22, 11]")
```

Output:

Output

```
Original/Sampled Array: [64, 25, 12, 22, 11]
Sorted Array: [11, 12, 22, 25, 64]

=== Code Execution Successful ===
```

---

5. Debugging Analysis

The main error in the given Bubble Sort code is the incorrect placement and handling of the swapped flag.

The statement swapped = False is initialized only once before the outer loop. It should be reset at the beginning of every pass so that the flag represents whether a swap occurred during the current pass.

Another error is the break statement inside the swapping condition. When the first swap occurs, break immediately stops the inner loop. This prevents the algorithm from comparing the remaining adjacent elements and may leave the array unsorted.

The break statement should be removed from the inner loop. Instead, after completing the entire pass, the program should check:

Optimization

The main optimization is the **swap flag**:

```
if not swapped:
  break
```

If an entire pass occurs without any swaps, no adjacent elements are out of order. Therefore, the array is already sorted and further passes are unnecessary.

This improves the best-case performance from **$O(n^2)$ to $O(n)$**.

6.Time Complexity

Case	**Complexity**
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

Space Complexity: O(1)

Best Case Example

Input: [1, 2, 3, 4, 5]

First pass:

- No swaps occur.
- swapped remains False.
- Algorithm terminates immediately.

Therefore:

Best Case = O(n)

7. Algorithm Steps

1. Start with the input array.
2. Set swapped = False at the beginning of each pass.
3. Compare adjacent elements.
4. If the left element is greater than the right element, swap them.
5. Set swapped = True whenever a swap occurs.
6. Continue comparing all required adjacent elements.
7. After completing the pass, check the swapped flag.
8. If swapped is False, terminate because the array is already sorted.
9. Otherwise, perform the next pass.
10. Return the sorted array.

8. Result Analysis

For the input:

[64, 25, 12, 22, 11]

The corrected Bubble Sort produces:

[11, 12, 22, 25, 64]

The corrected implementation successfully sorts the array in ascending order. Resetting the `swapped` flag at the beginning of each pass ensures that the flag represents only the current pass. Removing the incorrect `break` allows every required comparison to be performed. The early-termination condition avoids unnecessary passes when the array becomes sorted.

9.Conclusion:

The debugging process identifies that the main problem is the incorrect control of the swapped flag and the premature break statement. By resetting swapped for every outer-loop iteration and checking it only after a complete pass, Bubble Sort can terminate early when the array is already sorted. The corrected algorithm has O(n) best-case time complexity, $O(n^2)$ average and worst-case time complexity, and O(1) auxiliary space. This makes the implementation clearer and more practically efficient.